

Amrita Vishwa Vidyapeetham - School of Computing, Chennai

23DSA203 / 23CSE203: Data Structures and Algorithms

Topic: **Stack** (10 Challenge Programs) | Roll No: **CH.SC.U4CSE25262**

Question 1: Game of Two Lists

Problem: Two players A and B are given a list of n numbers. A picks from start and B picks from end. Compare elements: if A > B output 1 (B removes element), if A < B output 2 (A removes element), if equal output 0 (both remove elements). Repeat until empty.

Sample I/O: Input: 3 \n 1 2 3 | Output: 2 2 0

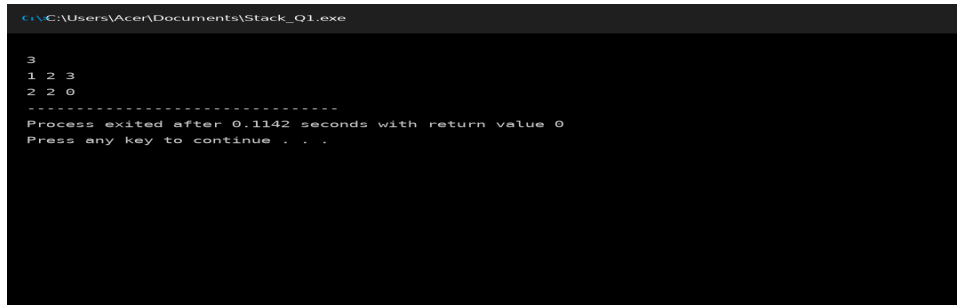
Source Code (C):

```
#include <stdio.h>

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;
    int a[100];
    for (int k = 0; k < n; k++) {
        scanf("%d", &a[k]);
    }

    int i = 0, j = n - 1;
    while (i < n && j >= 0) {
        if (a[i] > a[j]) {
            printf("1 ");
            j--;
        } else if (a[i] < a[j]) {
            printf("2 ");
            i++;
        } else {
            printf("0 ");
            i++;
            j--;
        }
    }
    printf("\n");
    return 0;
}
```

Execution Output:



```
C:\Users\Acer\Documents\Stack_Q1.exe

3
1 2 3
2 2 0
-----
Process exited after 0.1142 seconds with return value 0
Press any key to continue . . .
```

Question 2: Merge Two Linked-List Stacks

Problem: Given two stacks implemented using linked lists, merge stack 2 to the bottom of stack 1 and display the resulting elements from top to bottom.

Sample I/O: Input: 5 8 \n 9 8 7 6 5 \n 5 6 7 8 3 11 2 3 | Output: 5 6 7 8 9 3 2 11 3 8 7 6 5

Source Code (C):

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node* next;
} Node;

typedef struct mystack {
    Node* top;
} mystack;

mystack* createStack() {
    mystack* ms = (mystack*)malloc(sizeof(mystack));
    ms->top = NULL;
    return ms;
}

void push(int data, mystack* ms) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    newNode->data = data;
    newNode->next = ms->top;
    ms->top = newNode;
}

int pop(mystack* ms) {
    if (ms->top == NULL) return -1;
    Node* temp = ms->top;
    int val = temp->data;
    ms->top = ms->top->next;
    free(temp);
    return val;
}

void merge(mystack* ms1, mystack* ms2) {
    if (ms1->top == NULL) {
        ms1->top = ms2->top;
        return;
    }
    Node* curr = ms1->top;
    while (curr->next != NULL) {
        curr = curr->next;
    }
    curr->next = ms2->top;
}

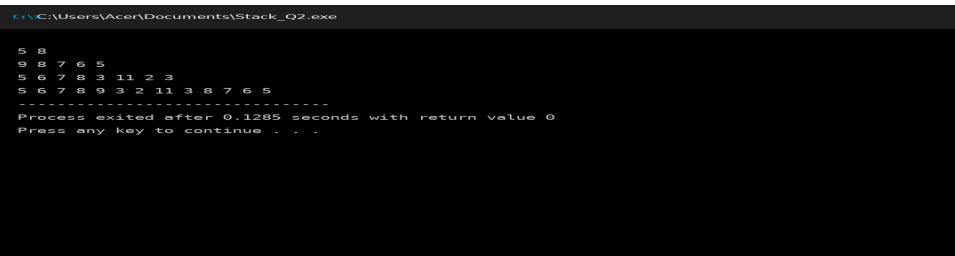
void display(mystack* ms) {
    Node* curr = ms->top;
    while (curr != NULL) {
        printf("%d ", curr->data);
        curr = curr->next;
    }
    printf("\n");
}

int main() {
    int n1, n2, val;
    if (scanf("%d %d", &n1, &n2) != 2) return 0;
    mystack* ms1 = createStack();
    mystack* ms2 = createStack();

    for (int i = 0; i < n1; i++) {
        scanf("%d", &val);
        push(val, ms1);
    }
    for (int i = 0; i < n2; i++) {
        scanf("%d", &val);
        push(val, ms2);
    }

    merge(ms1, ms2);
    display(ms1);
    return 0;
}
```

Execution Output:



```
C:\C:\Users\Acer\Documents\Stack_Q2.exe
5 8
9 8 7 6 5
5 6 7 8 3 11 2 3
.....
Process exited after 0.1285 seconds with return value 0
Press any key to continue . . .
```

Question 3: Next Greater Element Queries

Problem: Given an array of size n and q 1-based index queries, find the next strictly greater element to the right for each queried position. If no greater element exists, output -1.

Sample I/O: Input: 5 5 \n 62 70 28 62 92 \n 1 5 3 4 2 | Output: 2 -1 4 -1 -1

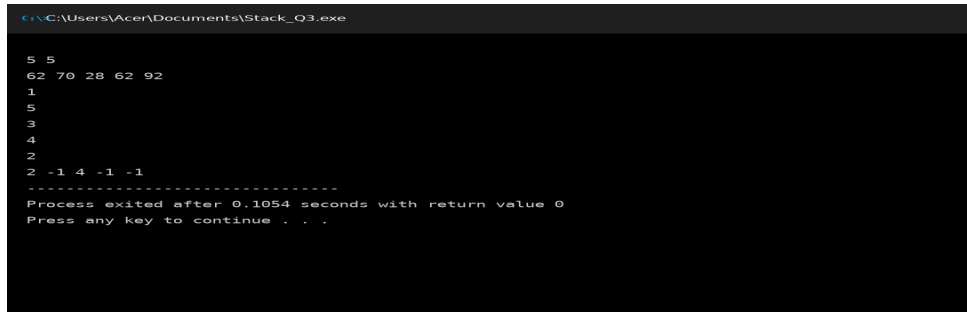
Source Code (C):

```
#include <stdio.h>

int main() {
    int n, q;
    if (scanf("%d %d", &n, &q) != 2) return 0;
    int arr[100];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    for (int k = 0; k < q; k++) {
        int idx;
        scanf("%d", &idx);
        int ans = -1;
        int x = idx - 1;
        for (int y = x + 1; y < n; y++) {
            if (arr[x] < arr[y]) {
                ans = arr[y];
                break;
            }
        }
        printf("%d ", ans);
    }
    printf("\n");
    return 0;
}
```

Execution Output:



```
C:\Users\Acer\Documents\Stack_Q3.exe

5 5
62 70 28 62 92
1
5
3
4
2
2 -1 4 -1 -1
-----
Process exited after 0.1054 seconds with return value 0
Press any key to continue . . .
```

Question 4: Implement Two Stacks in a Single Array

Problem: Implement two stacks stack1 and stack2 within a single array of size N such that stack1 grows from the left (index 0) and stack2 grows from the right (index N-1) to utilize space efficiently without overflow.

Sample I/O: Input: 1 2 3 4 5 | Output: Popped element from stack1 is:5 \n Popped element from stack2 is:4

Source Code (C):

```
#include <stdio.h>

#define SIZE 20
int arr[SIZE];
int top1 = -1;
int top2 = SIZE;

void push1(int x) {
    if (top1 < top2 - 1) {
        arr[++top1] = x;
    }
}

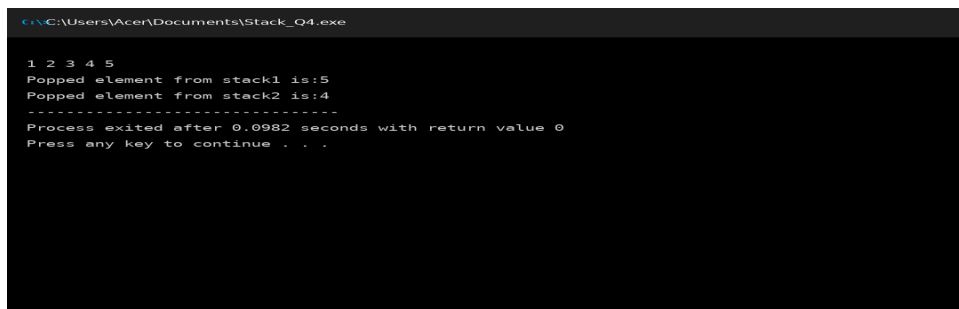
void push2(int x) {
    if (top1 < top2 - 1) {
        arr[--top2] = x;
    }
}

int pop1() {
    if (top1 >= 0) {
        return arr[top1--];
    }
    return -1;
}

int pop2() {
    if (top2 < SIZE) {
        return arr[top2++];
    }
    return -1;
}

int main() {
    int v1, v2, v3, v4, v5;
    if (scanf("%d %d %d %d %d", &v1, &v2, &v3, &v4, &v5) == 5) {
        push1(v1);
        push1(v2);
        push1(v5);
        push2(v3);
        push2(v4);
        printf("Popped element from stack1 is:%d\n", pop1());
        printf("Popped element from stack2 is:%d\n", pop2());
    }
    return 0;
}
```

Execution Output:



```
C:\C:\Users\Acer\Documents\Stack_Q4.exe

1 2 3 4 5
Popped element from stack1 is:5
Popped element from stack2 is:4
-----
Process exited after 0.0982 seconds with return value 0
Press any key to continue . . .
```

Question 5: Balanced Parentheses Checker

Problem: Given a string containing brackets '(', ')', '{', '}', '[', ']', verify if the brackets exist in valid matching pairs and close in the correct order using a stack.

Sample I/O: Input: `{()}]` | Output: `Balanced`

Source Code (C):

```
#include <stdio.h>
#include <string.h>

char stack[1000];
int top = -1;

void push(char c) {
    stack[++top] = c;
}

char pop() {
    if (top == -1) return '\0';
    return stack[top--];
}

int empty() {
    return top == -1;
}

int isMatching(char open, char close) {
    if (open == '(' && close == ')') return 1;
    if (open == '{' && close == '}') return 1;
    if (open == '[' && close == ']') return 1;
    return 0;
}

int main() {
    char str[1000];
    while (scanf("%s", str) == 1) {
        top = -1;
        int balanced = 1;
        for (int i = 0; str[i] != '\0'; i++) {
            char ch = str[i];
            if (ch == '(' || ch == '{' || ch == '[') {
                push(ch);
            } else if (ch == ')' || ch == '}' || ch == ']') {
                if (empty() || !isMatching(pop(), ch)) {
                    balanced = 0;
                    break;
                }
            }
        }
        if (balanced && empty()) {
            printf("Balanced\n");
        } else {
            printf("Not Balanced\n");
        }
    }
    return 0;
}
```

Execution Output:

```
C:\C:\Users\Acer\Documents\Stack_Q5.exe

{()}[]
{(())}
Balanced
Not Balanced
-----
Process exited after 0.1021 seconds with return value 0
Press any key to continue . . .
```

Question 6: Stock Span Problem

Problem: Given daily stock prices for n days, calculate the span of stock price for each day. The span $S[i]$ is defined as the maximum consecutive days just before the current day where the price was less than or equal to $price[i]$.

Sample I/O: Input: 6\n9 8 7 6 5 4 | Output: 1 1 1 1 1 1

Source Code (C):

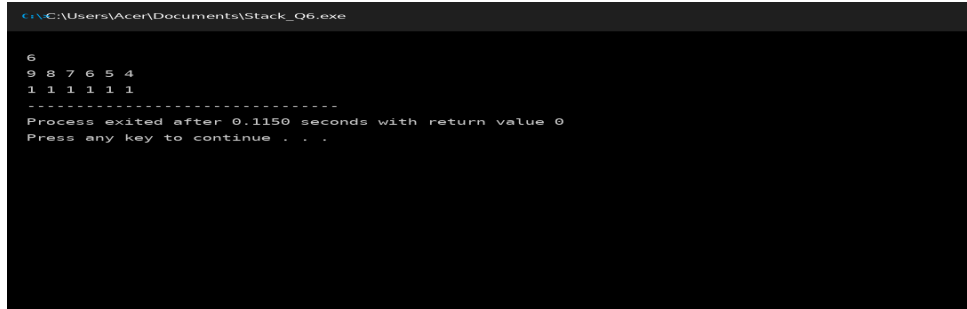
```
#include <stdio.h>

void calculateSpan(int price[], int n, int S[]) {
    S[0] = 1;
    for (int i = 1; i < n; i++) {
        S[i] = 1;
        for (int j = i - 1; (j >= 0) && (price[i] >= price[j]); j--) {
            S[i]++;
        }
    }
}

void printArray(int arr[], int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

int main() {
    int n;
    while (scanf("%d", &n) == 1) {
        int price[100], S[100];
        for (int i = 0; i < n; i++) {
            scanf("%d", &price[i]);
        }
        calculateSpan(price, n, S);
        printArray(S, n);
    }
    return 0;
}
```

Execution Output:



```
C:\C:\Users\Acer\Documents\Stack_Q6.exe

6
9 8 7 6 5 4
1 1 1 1 1 1
-----
Process exited after 0.1150 seconds with return value 0
Press any key to continue . . .
```

Question 7: Next Smaller of Next Greater Element

Problem: Given an array of n integers, for each element find the next greater element to its right, and then find the next smaller element to the right of that greater element.

Sample I/O: Input: 8 \n 3 7 1 7 8 4 5 2 | Output: 1 4 4 4 -1 2 -1 -1

Source Code (C):

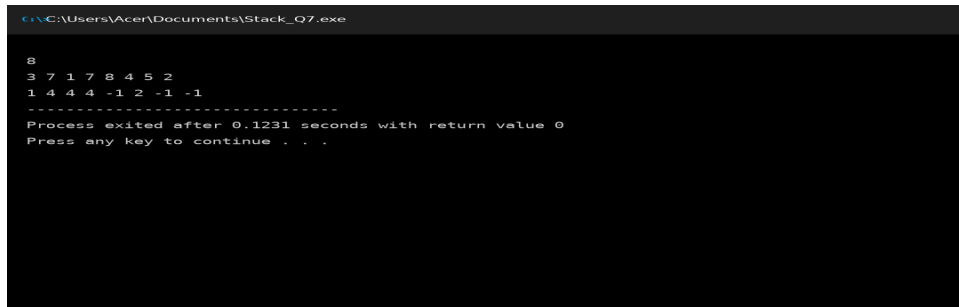
```
#include <stdio.h>

int main() {
    int n;
    while (scanf("%d", &n) == 1) {
        int a[100];
        for (int i = 0; i < n; i++) {
            scanf("%d", &a[i]);
        }

        for (int i = 0; i < n; i++) {
            int next_greater_idx = -1;
            for (int j = i + 1; j < n; j++) {
                if (a[j] > a[i]) {
                    next_greater_idx = j;
                    break;
                }
            }

            int ans = -1;
            if (next_greater_idx != -1) {
                for (int k = next_greater_idx + 1; k < n; k++) {
                    if (a[k] < a[next_greater_idx]) {
                        ans = a[k];
                        break;
                    }
                }
            }
            printf("%d ", ans);
        }
        printf("\n");
    }
    return 0;
}
```

Execution Output:



```
C:\Users\Acer\Documents\Stack_Q7.exe
8
3 7 1 7 8 4 5 2
1 4 4 4 -1 2 -1 -1
-----
Process exited after 0.1231 seconds with return value 0
Press any key to continue . . .
```

Question 8: Maximum Bitwise XOR in Array

Problem: Given an array of n integers, find the maximum bitwise XOR value achievable between any pair of elements ($\text{arr}[i] \oplus \text{arr}[j]$) where $i < j$.

Sample I/O: Input: 5 \n 1 2 3 8 6 | Output: 11

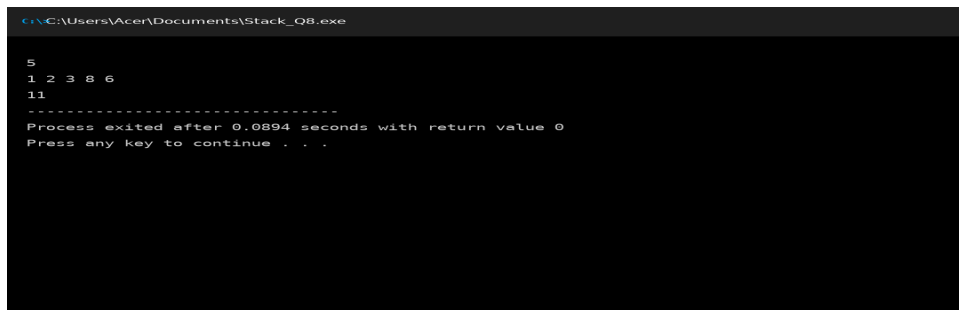
Source Code (C):

```
#include <stdio.h>

int main() {
    int n;
    while (scanf("%d", &n) == 1) {
        int arr[100];
        int st[100];
        for (int i = 0; i < n; i++) {
            scanf("%d", &arr[i]);
        }

        int max_val = 0;
        for (int i = 0; i < n; i++) {
            for (int j = i + 1; j < n; j++) {
                if (arr[i] < arr[j]) {
                    st[i] = arr[i] ^ arr[j];
                    if (st[i] > max_val) {
                        max_val = st[i];
                    }
                }
            }
        }
        printf("%d\n", max_val);
    }
    return 0;
}
```

Execution Output:



```
C:\Users\Acer\Documents\Stack_Q8.exe

5
1 2 3 8 6
11
-----
Process exited after 0.0894 seconds with return value 0
Press any key to continue . . .
```

Question 9: Stack Operations via Queue Simulation

Problem: Simulate standard stack push and pop operations using a queue data structure, and display the top element of the stack after insertions and specified deletions.

Sample I/O: Input: 5 2 \n 1 2 3 4 5 | Output: top of element 5 \n top of element 3

Source Code (C):

```
#include <stdio.h>

int q[100];
int front = 0, rear = -1;

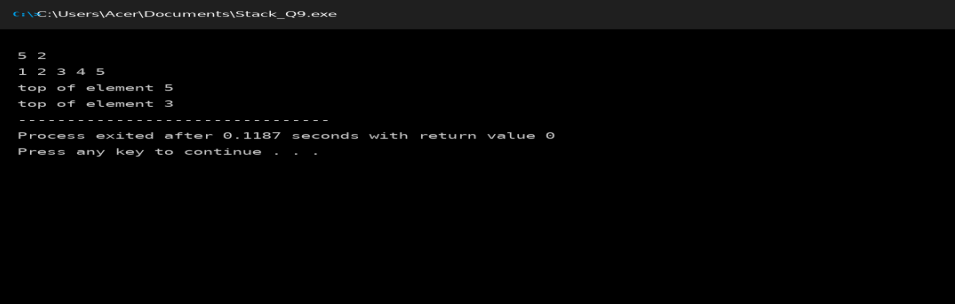
void push(int val) {
    q[++rear] = val;
}

int pop() {
    if (rear >= front) {
        return q[rear--];
    }
    return -1;
}

int top() {
    if (rear >= front) {
        return q[rear];
    }
    return -1;
}

int main() {
    int n, pops;
    while (scanf("%d %d", &n, &pops) == 2) {
        front = 0; rear = -1;
        for (int i = 0; i < n; i++) {
            int val;
            scanf("%d", &val);
            push(val);
        }
        printf("top of element %d\n", top());
        for (int i = 0; i < pops; i++) {
            pop();
        }
        printf("top of element %d\n", top());
    }
    return 0;
}
```

Execution Output:



```
C:\C:\Users\Acer\Documents\Stack_Q9.exe

5 2
1 2 3 4 5
top of element 5
top of element 3
-----
Process exited after 0.1187 seconds with return value 0
Press any key to continue . . .
```

Question 10: Postfix to Prefix Expression Conversion

Problem: Given a mathematical expression in Postfix (Reverse Polish) notation, convert it into an equivalent Prefix (Polish) notation string using a stack of operands.

Sample I/O: Input: ABC/-AK/L-* | Output: *-A/BC-/AKL

Source Code (C):

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

char stack[100][100];
int top = -1;

void push(char* str) {
    strcpy(stack[++top], str);
}

void pop(char* res) {
    strcpy(res, stack[top--]);
}

void postToPre(char* post_exp) {
    top = -1;
    int len = strlen(post_exp);

    for (int i = 0; i < len; i++) {
        char ch = post_exp[i];
        if (isalnum(ch)) {
            char op[2] = {ch, '\0'};
            push(op);
        } else {
            char op2[100], op1[100];
            pop(op2);
            pop(op1);

            char temp[100];
            temp[0] = ch;
            temp[1] = '\0';
            strcat(temp, op1);
            strcat(temp, op2);
            push(temp);
        }
    }
    printf("%s\n", stack[top]);
}

int main() {
    char exp[100];
    while (scanf("%s", exp) == 1) {
        postToPre(exp);
    }
    return 0;
}
```

Execution Output:

```
C:\C:\Users\Acer\Documents\Stack_Q10.exe

ABC/-AK/L-*
AB+CD-*
*-A/BC-/AKL
*+AB-CD
-----
Process exited after 0.1342 seconds with return value 0
Press any key to continue . . .
```